Midterm Review: Definitions and Problems

Lecture 24

CS 4880 Kirk Duran

Today Is Practice in Recognizing the Right Machine

The midterm is cumulative, but most questions begin with the same move: identify what kind of reasoning problem you have been given.

Search	Which reachable path or state should we pursue?
Games	Which action survives an opponent's best response?
Logic	What conclusion follows from stated facts and rules?
Probability	What should evidence make us believe?
Filtering	What hidden state best explains motion and measurements?
MDPs	Which action has the best long-run consequences?

Key idea

Do not start by hunting for a formula. Start by naming the representation, the quantity being computed, and the assumptions that make the computation valid.

A Compact Definition Map

Search and games

- $g(n)$: cost incurred so far.
- $h(n)$: estimated remaining cost.
- $f(n) = g(n) + h(n)$: A* priority.
- **Minimax**: MAX chooses assuming MIN responds optimally.
- **UCB1**: balances high observed reward and underexplored actions in MCTS.

Reasoning under uncertainty

- **Conditional probability**: $P(A \mid B)$, a probability after observing B.
- **Bayes' rule**: prior $\times$ likelihood, then normalize.
- **Belief state**: probability distribution over hidden states.
- **MDP policy**: a rule mapping states to actions.

Key idea

A definition question is not a vocabulary contest. A strong answer says what the object represents, how it is used, and what distinguishes it from nearby concepts.

A*: Spend Cost So Far Plus a Forecast of What Remains

A* orders the frontier by

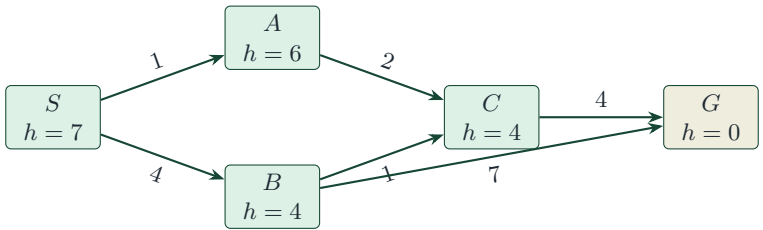
$$f(n) = g(n) + h(n).$$

- $g(n)$ is exact path cost from the start to n.
- $h(n)$ estimates cost from n to a goal.
- An **admissible** heuristic never overestimates the true remaining cost.
- With the usual graph-search conditions, admissibility lets A* return an optimal path.

Key idea

Uniform-cost search is the special case $h(n) = 0$. Greedy best-first search uses only $h(n)$; A* must keep both past cost and estimated future cost in view.

A* Problem: Which Node Leaves the Frontier Next?



Trace graph search from S. On every frontier entry, write (g, h, f).

In class

After expanding S, which node is expanded next: A or B? Do not answer from the picture; compute f for both.

A* Solution: The Frontier Is the Whole Story

Expand	New or improved frontier entries	Next choice
S	$A : (1, 6, 7),\ B : (4, 4, 8)$	A
A	$C : (3, 4, 7),\ B : (4, 4, 8)$	C
C	$G : (7, 0, 7),\ B : (4, 4, 8)$	G
G	goal reached	stop

The returned path is

$$S \to A \to C \to G, \qquad \text{cost } 7.$$

Key idea

The heuristic did not make B disappear; it only made A and then C more promising. A* still compares complete $f = g + h$ priorities, not intuition about which drawn route looks shorter.

Minimax: Your Opponent Does Not Cooperate

In a deterministic, two-player, zero-sum game with perfect information:

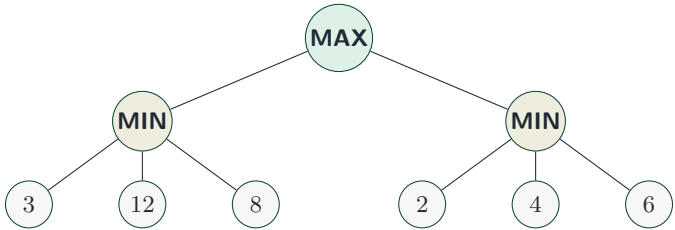
- MAX chooses the child with the greatest backed-up value.
- MIN chooses the child with the smallest backed-up value.
- Leaf values are terminal utilities or heuristic estimates at a depth limit.

$$V_{\mathrm{MAX}}(s) = \max_{a \in Actions(s)} V(Result(s,a)), \qquad V_{\mathrm{MIN}}(s) = \min_{a \in Actions(s)} V(Result(s,a)).$$

Key idea

Minimax does not predict the opponent's most likely action. It protects against their best action from their own point of view.

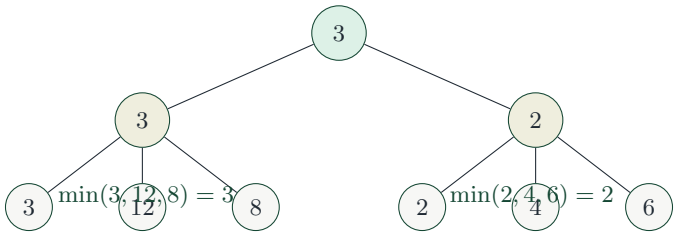
Minimax Problem: Back Up the Leaves



In class

Work bottom-up. What does each MIN node return to the root? Which root action is selected? Explain why MAX cannot justify choosing the left child because one of its leaves is 12.

Minimax Solution: Every Player Optimizes Their Own Node



$$\max(3, 2) = 3.$$

MAX chooses the **left** action. The 12 is irrelevant because MIN, if given that subtree, chooses the 3 instead.

Key idea

Alpha-Beta pruning changes how many nodes we inspect, not the minimax value we eventually return.

Forward Chaining: Make Every Rule Instance Whose Premises Are Known

- A knowledge base contains facts and rules.
- **Forward chaining** begins with known facts and repeatedly applies rules whose premises match those facts.
- **Unification** finds a substitution that makes an abstract rule premise match a concrete fact.
- **Generalized modus ponens** adds the rule's instantiated conclusion.

Key idea

Forward chaining is data-driven: new facts trigger the rules that can use them. It is especially natural when you want every conclusion supported by the currently known data.

Forward-Chaining Problem: What Can We Derive About Mina?

Rules

$$\begin{aligned} \mathit{Bird}(x) &\Rightarrow \mathit{Animal}(x) \\ \mathit{Animal}(x) \wedge \mathit{HasWings}(x) &\Rightarrow \mathit{CanFly}(x) \\ \mathit{CanFly}(x) \wedge \mathit{Healthy}(x) &\Rightarrow \mathit{Cleared}(x) \end{aligned}$$

Initial facts

$$\mathit{Bird}(\mathit{Mina}), \qquad \mathit{HasWings}(\mathit{Mina}), \qquad \mathit{Healthy}(\mathit{Mina}).$$

Apply forward chaining until no new fact can be produced. Record the substitution whenever a rule fires.

Forward-Chaining Solution: New Facts Unlock Later Rules

1. Match *Bird*(x) to *Bird*(*Mina*) with
$$\{x/\mathit{Mina}\}.$$
Add *Animal*(*Mina*).
2. Now *Animal*(*Mina*) and *HasWings*(*Mina*) satisfy the second rule with the same substitution. Add *CanFly*(*Mina*).
3. Finally, *CanFly*(*Mina*) and *Healthy*(*Mina*) satisfy the third rule. Add *Cleared*(*Mina*).

Key idea

The conclusion of one inference can become a premise for another. The process stops at a fixed point: no rule can add a fact that is not already known.

Naive Bayes: Compare Classes with Prior Times Likelihood

For class C and observed features $x_1, \ldots, x_k$:

$$P(C \mid x_1, \ldots, x_k) \propto P(C) \prod_i P(x_i \mid C).$$

The proportionality sign means we may first compute unnormalized class scores and normalize only if a probability is required.

Key idea

Naive Bayes assumes features are conditionally independent *given the class*. The assumption is usually not literally true; the tradeoff is a compact model and cheap inference.

Naive Bayes Problem: Classify an Email with Two Words

Let S mean spam and $\neg S$ mean not spam.

$$P(S) = 0.40, \qquad P(\neg S) = 0.60.$$

	S	$\neg S$
$P(\textit{free} \mid \text{class})$	0.80	0.10
$P(\textit{meeting} \mid \text{class})$	0.05	0.60

An email contains both *free* and *meeting*.

In class

Compute one score for spam and one for not spam. Then normalize to obtain $P(S \mid \textit{free}, \textit{meeting})$.

Naive Bayes Solution: The Words Pull in Opposite Directions

$$\text{score}(S) = 0.40(0.80)(0.05) = 0.016,$$
$$\text{score}(\neg S) = 0.60(0.10)(0.60) = 0.036.$$

Normalize with the total score $0.016 + 0.036 = 0.052$:

$$P(S \mid \textit{free}, \textit{meeting}) = \frac{0.016}{0.052} \approx 0.308.$$

Therefore predict $\neg S$.

Key idea

A feature can be strong evidence for spam without deciding the final label. Classification compares every class's prior-weighted explanation of *all* observed features.

Prior Sampling: Generate Complete Worlds in Topological Order

In **prior sampling**, generate each Bayesian-network variable after its parents have been sampled.



Suppose

$$P(R) = 0.30, \qquad P(W \mid R) = 0.90, \qquad P(W \mid \neg R) = 0.10.$$

Key idea

A prior sample is a synthetic complete assignment drawn from the model's joint distribution. Estimate a probability by the fraction of generated worlds in which the event occurs.

Prior-Sampling Problem: Estimate $P(W)$ from Samples

A sampling run produces these ten complete assignments:

sample	1	2	3	4	5	6	7	8	9	10
R	T	F	F	T	F	F	F	T	F	F
W	T	F	F	T	F	F	F	F	T	F

In class

Estimate $P(W)$ from this run. Then explain why the estimate is not required to equal the exact answer, even though every sample was generated correctly.

Prior-Sampling Solution: Count Worlds, Then Remember the Error Bar

Wet grass appears in samples 1, 4, and 9:

$$\widehat{P}(W) = \frac{3}{10} = 0.30.$$

The exact value from enumeration would be

$$P(W) = P(R)P(W \mid R) + P(\neg R)P(W \mid \neg R) = 0.30(0.90) + 0.70(0.10) = 0.34.$$

Key idea

Sampling trades exactness for scalability. With more independent samples, the estimate should converge toward the model's probability, but any finite run can be high or low by chance.

A One-Dimensional Kalman Filter Alternates Predict and Correct

We represent belief about position as a Gaussian:

$$x \sim \mathcal{N}(\mu, P).$$

- μ is the estimated position.
- P is uncertainty, represented here as variance.
- Motion changes the expected position and adds process uncertainty Q.
- A sensor reports z with measurement variance R.

For our simple position model:

$$\bar{\mu} = \mu + u, \qquad \bar{P} = P + Q,$$

$$K = \frac{\bar{P}}{\bar{P} + R}, \qquad \mu' = \bar{\mu} + K(z - \bar{\mu}), \qquad P' = (1 - K)\bar{P}.$$

Key idea

Kalman Problem: Move, Then Read One Noisy Sensor

A robot begins with

$$x_0 \sim \mathcal{N}(10, 1).$$

It commands a move of $u = 5$. Motion has variance $Q = 4$. After moving, a position sensor reports $z = 17$ with variance $R = 4$.

1. Predict the belief after motion: compute $\bar{\mu}$ and $\bar{P}$.
2. Compute the Kalman gain K.
3. Update the mean μ' and variance P' after seeing the sensor reading.

In class

Before calculating: should the posterior mean land at 15, at 17, or strictly between them? Why?

Kalman Solution, Step 1: Prediction Follows the Motion Model

The commanded move shifts the estimated position by five units:

$$\bar{\mu} = 10 + 5 = 15.$$

Motion noise increases uncertainty:

$$\bar{P} = 1 + 4 = 5.$$

Before reading the sensor, the belief is

$$x \sim \mathcal{N}(15, 5).$$

Key idea

A prediction is not merely "add the movement." It must also acknowledge that real motion is imperfect, so uncertainty grows before the measurement arrives.

Kalman Solution, Step 2: Correct Toward the Measurement

The gain is

$$K = \frac{5}{5+4} = \frac{5}{9} \approx 0.556.$$

The sensor is two units above the prediction, so move 55.6% of that gap:

$$\mu' = 15 + \frac{5}{9}(17 - 15) = 15 + \frac{10}{9} \approx 16.11.$$

The corrected variance is

$$P' = (1 - \tfrac{5}{9})(5) = \frac{20}{9} \approx 2.22.$$

Key idea

The posterior lies between prediction and measurement, and it is more certain than either alone. Equal-sized predicted and measurement variances would have produced $K = 0.5$.

MDPs: Choose for the Future, Not Just the Immediate Reward

A Markov Decision Process is commonly written

$$\langle S, A, T, R, \gamma \rangle.$$

- S: states; A: actions available in each state.
- $T(s, a, s')$: transition probability to next state s'.
- $R(s, a, s')$: immediate reward for that transition.
- γ: discount factor; a **policy** chooses an action in each state.

A one-step action-value backup is

$$Q(s, a) = \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma V(s') \right].$$

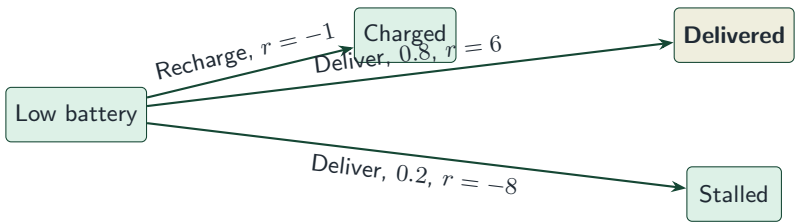
Key idea

MDP planning assumes the transition and reward model are known. Reinforcement learning instead estimates useful values from experience.

Small MDP Problem: Deliver Now or Recharge First?

At state *Low battery*, use $\gamma = 0.9$. Suppose later planning has supplied

$$V(\textit{Charged}) = 10, \qquad V(\textit{Delivered}) = 0, \qquad V(\textit{Stalled}) = -5.$$



Compute $Q(\textit{Low}, \textit{Recharge})$ and $Q(\textit{Low}, \textit{Deliver})$.

Small MDP Solution: Delayed Value Can Beat Immediate Progress

Recharge has one possible transition:

$$Q(\textit{Low}, \textit{Recharge}) = -1 + 0.9(10) = 8.$$

Deliver has two possible transitions:

$$\begin{aligned} Q(\textit{Low}, \textit{Deliver}) &= 0.8\,[6 + 0.9(0)] + 0.2\,[-8 + 0.9(-5)] \\ &= 4.8 - 2.5 = 2.3. \end{aligned}$$

The policy should choose **Recharge**, even though it has immediate reward -1.

Key idea

Greedy action selection sees $6 > -1$ and delivers immediately. MDP reasoning includes the next state's value and the risk of reaching a bad future state.

Exam Checklist: What Should You Write Before You Calculate?

1. **Search:** frontier entries and the priority rule (g, h, f).
2. **Games:** whose node it is and whether to take max, min, or an expectation.
3. **Logic:** facts, rule premises, and substitutions that make them match.
4. **Bayes:** prior, likelihood, competing hypotheses, then the normalizing denominator.
5. **Sampling:** what each sample represents and what fraction or weighted fraction estimates the query.
6. **Filtering:** predicted mean/variance before the sensor, then gain and corrected belief.
7. **MDPs:** next states, their probabilities, immediate rewards, discount, and future values.

Key idea

Most lost points come from silently switching representations: treating a likelihood as a posterior, a heuristic as true cost, a reward as a return, or a leaf value as a policy value.

Closing: Seven Problems, One Habit

Every technique today uses a representation to make a difficult reasoning problem tractable.

A*	cost-so-far plus admissible estimate
Minimax	alternating optimal choices
Forward chaining	facts and rule instances
Naive Bayes	prior-weighted, conditionally independent evidence
Prior sampling	complete worlds drawn from a model
Kalman filtering	Gaussian belief updated by noisy motion and sensing
MDPs	stochastic actions plus long-run value

Key idea

On the midterm, name the representation first. Then carry the semantics of each number all the way through the calculation.